

Barbería Cale utiliza una arquitectura cliente-servidor. El cliente Android/web presenta la interfaz y consume una API REST; el backend centraliza autenticación, autorización y reglas de negocio; y PostgreSQL en Supabase conserva los datos de usuarios y citas. Esta organización evita que las pantallas accedan directamente a la base de datos.

Arquitectura cliente-servidor de Barbería Cale v1.0.

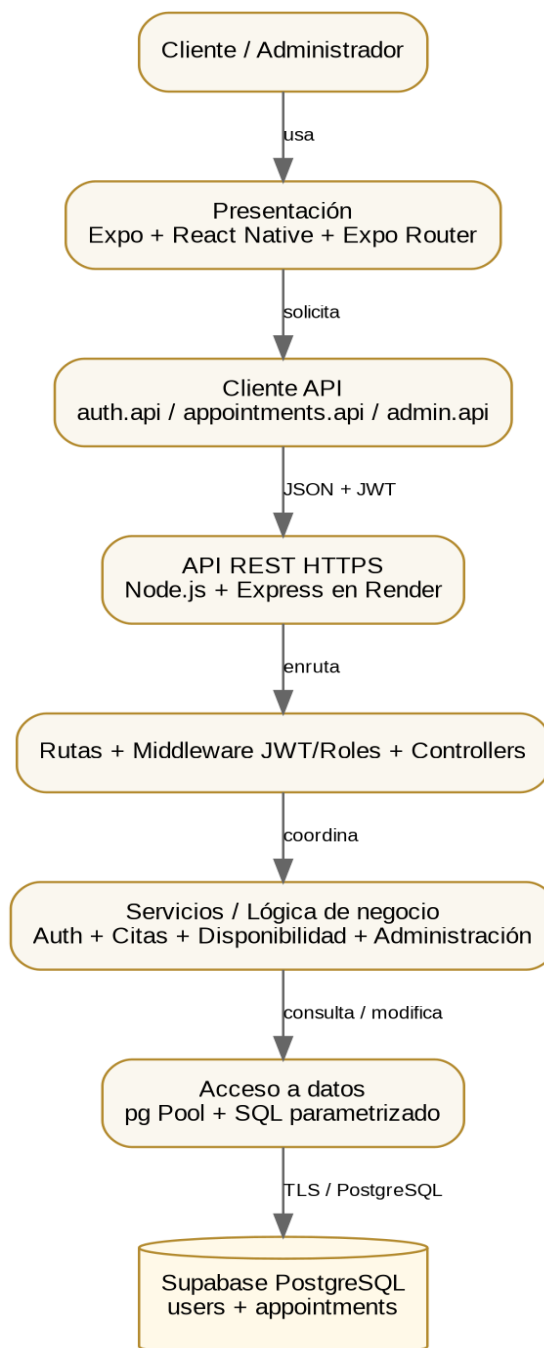
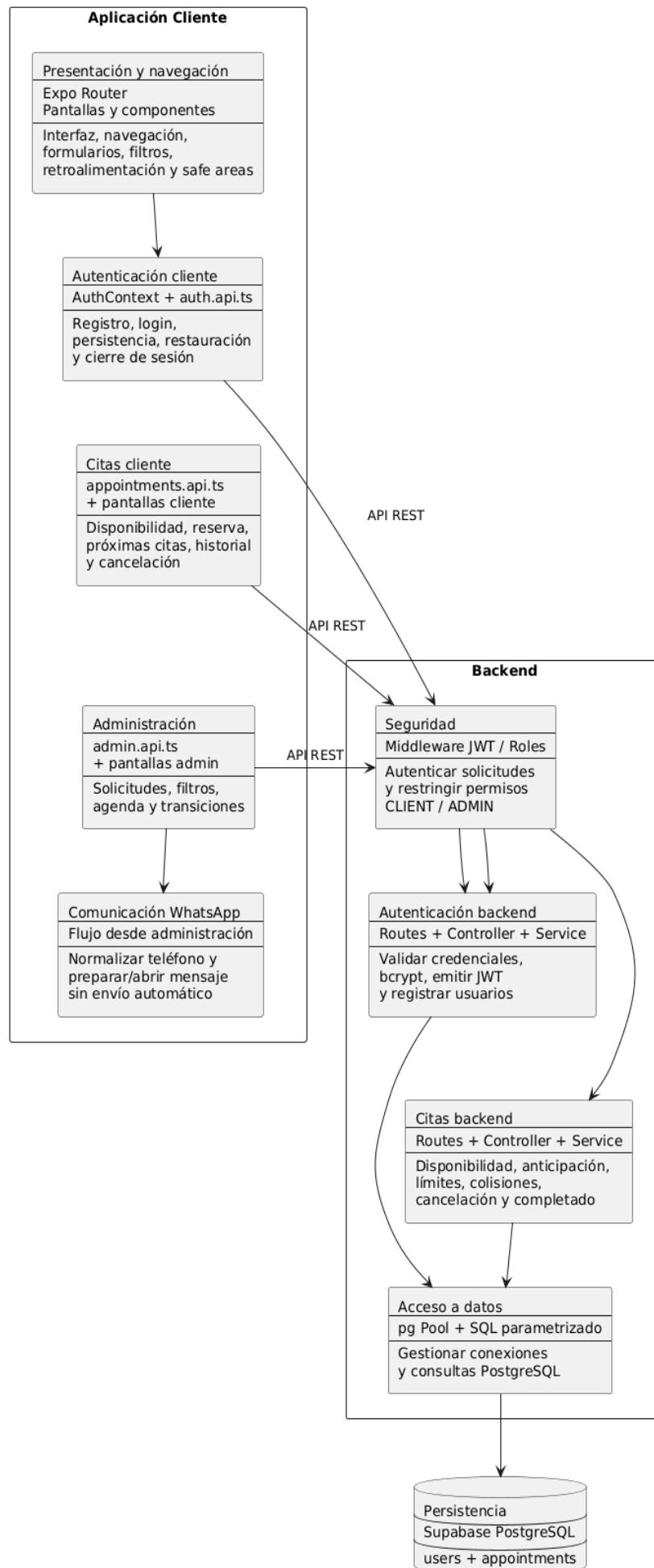
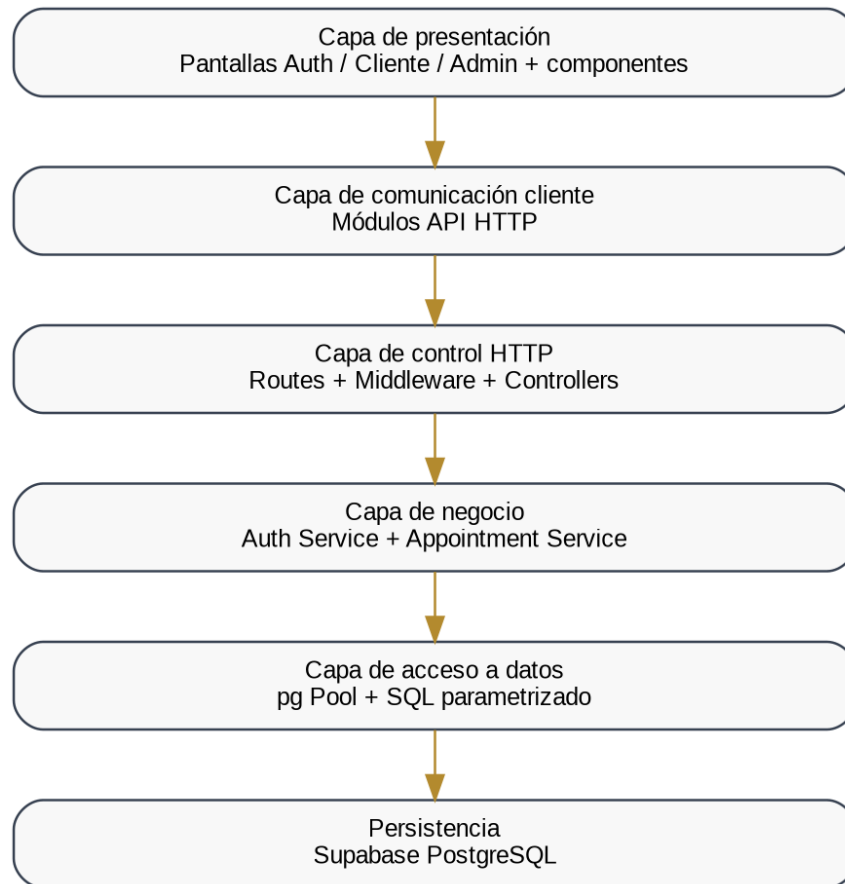


Diagrama de componentes y módulos - Barbería Cale v1.0



1. Separación por capas



Separación de responsabilidades por capas.

Justificación de las decisiones arquitectónicas

1. Arquitectura cliente-servidor

Barbería Cale utiliza una arquitectura cliente-servidor porque el sistema posee dos tipos de usuario —cliente y administrador— que deben consultar y modificar una misma fuente de información. Una cita creada por un cliente debe ser visible para el administrador, y cualquier cambio de estado realizado por el administrador debe reflejarse posteriormente en la aplicación del cliente. Mantener los datos y reglas únicamente en cada dispositivo produciría inconsistencias y duplicaría lógica. Por esta razón, el backend se establece como autoridad central del sistema y la aplicación cliente actúa como interfaz de acceso. Esta decisión también permite que Android y web compartan las mismas reglas de negocio y la misma información sin implementar soluciones independientes. Para el alcance de una barbería pequeña, esta arquitectura resulta suficientemente robusta sin introducir la complejidad de una infraestructura distribuida más avanzada.

2. Separación por capas y responsabilidades

La solución se organiza mediante separación por capas para evitar que la interfaz, la lógica de negocio, la seguridad y el acceso a datos queden concentrados en un mismo componente. En el cliente se mantienen principalmente la presentación, navegación, estado de autenticación y comunicación con la API. En el backend, las rutas identifican los endpoints, los middleware aplican controles transversales, los controladores gestionan la interacción HTTP, los servicios contienen las reglas del dominio y la capa de datos administra PostgreSQL. Esta organización reduce el acoplamiento y facilita identificar qué componente debe modificarse cuando cambia una regla o funcionalidad. Para un proyecto del tamaño de Barbería Cale, ofrece un equilibrio entre buena arquitectura y capacidad técnica del equipo: es suficientemente modular para demostrar mantenibilidad y trazabilidad, pero evita la sobrecarga de arquitecturas como

microservicios, que no serían justificables por el volumen funcional ni el tráfico esperado de la primera versión.

3. Capa de presentación: Expo, React Native y TypeScript

Para la interfaz se seleccionó Expo con React Native y TypeScript. React Native permite reutilizar una parte considerable del mismo código en Android y web, lo que se adapta directamente al alcance multiplataforma del proyecto. Desarrollar dos aplicaciones separadas implicaría duplicar interfaces, validaciones, mantenimiento y pruebas, aumentando el tiempo y costo del prototipo sin una ganancia proporcional. Expo simplifica la configuración del proyecto, la ejecución en dispositivos físicos y la generación de compilaciones Android, reduciendo la cantidad de configuración nativa necesaria durante el desarrollo académico. TypeScript añade tipado estático, lo cual ayuda a detectar errores antes de ejecución y resulta especialmente útil al manejar usuarios, citas, respuestas de API, estados y estructuras compartidas. Esta combinación prioriza productividad y compatibilidad sin impedir que en el futuro se acceda a configuración nativa cuando sea necesario.

4. Navegación con Expo Router

La navegación se implementa con Expo Router porque su estructura basada en archivos se adapta de forma natural a la organización de Barbería Cale: rutas públicas, autenticación, cliente y administración. Esto facilita mantener layouts diferentes por rol, localizar pantallas y relacionarlas con los requisitos funcionales. Al pertenecer al mismo ecosistema de Expo, evita introducir dependencias adicionales sin una necesidad concreta. También favorece la futura ampliación del proyecto porque nuevas pantallas pueden incorporarse dentro de una estructura de rutas ya definida, manteniendo la separación entre áreas funcionales.

5. Módulos API en el cliente

La comunicación HTTP se encapsula en módulos como `auth.api.ts`, `appointments.api.ts` y `admin.api.ts`. Esta decisión evita que cada pantalla conozca directamente las rutas del servidor, formatos HTTP y detalles de implementación de la API. Las pantallas se concentran en la experiencia de usuario, mientras los módulos API se encargan de enviar y recibir información. Esto reduce duplicación de código y facilita cambios futuros; por ejemplo, si cambia una URL, estructura de respuesta o endpoint, la modificación puede realizarse en un punto central en lugar de alterar múltiples pantallas. También mejora la capacidad de prueba y mantenimiento del cliente.

6. Backend con Node.js y Express

Para el servidor se seleccionó Node.js con Express porque las necesidades del sistema son principalmente transaccionales: recibir solicitudes, validar usuarios, consultar disponibilidad, aplicar reglas de reservas y realizar operaciones sobre una base de datos. No existe un requerimiento de procesamiento intensivo que justifique tecnologías más complejas. Express proporciona una solución ligera y suficientemente flexible para construir una API REST con rutas, middleware y controladores. Además, utilizar JavaScript en el backend y JavaScript/TypeScript en el frontend reduce la cantidad de lenguajes y paradigmas que el equipo debe administrar, algo importante en un prototipo académico donde las decisiones deben ser implementables, comprensibles y defendibles. Desde el punto de vista de costo, Node.js y Express son de código abierto y ampliamente soportados por plataformas de hosting de bajo costo.

7. Organización interna del backend

El backend se divide en rutas, middleware, controladores, servicios y acceso a datos. Las rutas determinan qué endpoint atiende cada solicitud; los middleware verifican autenticación y roles; los controladores coordinan la respuesta HTTP; los servicios implementan reglas como anticipación mínima, límites de citas, disponibilidad y cambios de estado; y el acceso a datos concentra las consultas SQL. Esta decisión es especialmente importante porque las reglas críticas no deben depender de la interfaz. Un usuario podría modificar o sustituir el cliente, por lo que las restricciones reales deben permanecer en el servidor. Esto garantiza que Android y web reciban el mismo comportamiento y mejora simultáneamente seguridad, consistencia y mantenibilidad.

8. API REST como frontera del sistema

La API REST funciona como frontera entre presentación y lógica de negocio. Se seleccionó porque las principales entidades del proyecto —usuarios, autenticación y citas— pueden representarse de forma clara mediante recursos y operaciones HTTP. REST resulta más sencillo de implementar, probar y documentar que alternativas más complejas para un proyecto de este alcance. El cliente solo necesita conocer el contrato de la API y no la forma en que se almacenan los datos internamente. Esto permite desacoplar frontend y backend y facilita una posible evolución futura; por ejemplo, podría desarrollarse posteriormente un panel administrativo web independiente consumiendo la misma API sin duplicar las reglas del negocio.

9. PostgreSQL como base de datos

Se eligió PostgreSQL porque los datos del proyecto son claramente estructurados y relacionales. Un usuario puede tener múltiples citas y cada cita pertenece a un usuario, lo que se representa naturalmente mediante claves primarias y foráneas. Además, el

sistema necesita proteger la integridad de los horarios. PostgreSQL permite utilizar restricciones, índices y relaciones para reforzar reglas que no deberían depender exclusivamente del código del backend. Esto es especialmente importante para evitar colisiones: aunque dos solicitudes concurrentes pasaran casi simultáneamente una validación lógica, una restricción en la base de datos puede impedir que ambas creen una cita activa para el mismo horario. Esta combinación de validación en backend más integridad en base de datos proporciona mayor fiabilidad.

10. Acceso a PostgreSQL con pg y SQL parametrizado

El backend utiliza pg y consultas SQL parametrizadas. Para el alcance actual, donde el modelo principal está compuesto por usuarios y citas, utilizar SQL directamente evita incorporar un ORM completo que aumentaría dependencias y abstracción sin una necesidad real. Las consultas permanecen visibles y controlables, lo cual también favorece el aprendizaje y la depuración. La parametrización separa los datos proporcionados por el usuario de la estructura de la sentencia SQL, ayudando a reducir riesgos de inyección. Si el dominio creciera significativamente, esta capa podría sustituirse o ampliarse sin necesidad de cambiar la API pública del sistema.

11. Supabase para persistencia en la nube

La base de datos PostgreSQL se aloja en Supabase porque permite disponer de una base administrada en la nube sin asumir el costo y mantenimiento de un servidor de base de datos propio. Para una primera versión con tráfico reducido, esto es adecuado económicamente y simplifica tareas como disponibilidad del servicio, conexiones y administración. Además, al utilizar PostgreSQL estándar, el proyecto no queda atado exclusivamente a Supabase: la misma estructura podría migrarse a otro proveedor si las

necesidades de capacidad, disponibilidad o costo cambiaran. Así, la decisión favorece el prototipo actual sin cerrar la posibilidad de escalabilidad futura.

12. Render para despliegue del backend

El backend se despliega en Render porque ofrece una forma sencilla de publicar una aplicación Node.js sin administrar directamente servidores, redes o sistemas operativos. Esto permite concentrar el esfuerzo en el producto y no en infraestructura que excede el alcance académico del proyecto. Su integración con GitHub facilita que los cambios enviados al repositorio puedan desplegarse de forma reproducible. Para el volumen actual de Barbería Cale, un servicio de entrada resulta suficiente y económicamente razonable. Si el tráfico o los requisitos de disponibilidad aumentaran, el backend podría trasladarse posteriormente a infraestructura de mayor capacidad porque la aplicación utiliza tecnologías estándar y no depende estructuralmente de Render.

13. Escalabilidad y portabilidad de la infraestructura

La estrategia de escalabilidad no consiste en sobredimensionar la versión 1.0, sino en utilizar componentes que puedan crecer o reemplazarse. Express y PostgreSQL son tecnologías ampliamente compatibles con distintos proveedores. Por ello, si Barbería Cale evolucionara hacia múltiples barberos, más sucursales o un volumen mayor de usuarios, el backend podría desplegarse en servicios de mayor capacidad y la base PostgreSQL podría migrarse sin rediseñar necesariamente el frontend o la API. Esto reduce el riesgo de dependencia de proveedor y permite que el crecimiento se realice cuando exista una necesidad real, en lugar de asumir desde el inicio costos técnicos y económicos innecesarios.

14. Autenticación, bcrypt y JWT

La autenticación utiliza bcrypt para las contraseñas y JWT para las sesiones. Las contraseñas no deben almacenarse en texto plano, por lo que se transforman mediante hash antes de persistirse. JWT permite que el backend identifique al usuario y su rol en cada solicitud protegida. El control de autorización se mantiene en middleware del servidor, de modo que ocultar una opción en la interfaz no sea la única barrera de seguridad. Un usuario CLIENT que intentara acceder directamente a un endpoint administrativo debe ser rechazado por el backend. La persistencia opcional de sesión se mantiene separada en el cliente mediante AuthContext, conservando una clara división entre identidad, autorización y experiencia de usuario.

15. Git y GitHub

Git y GitHub se utilizan tanto por razones técnicas como académicas. Git permite conservar el historial de cambios, comparar versiones y recuperar estados anteriores. GitHub proporciona un repositorio remoto, facilita evidenciar la evolución del proyecto y permite relacionar commits con funcionalidades, correcciones y entregables. Esto mejora la trazabilidad del trabajo. Además, la conexión con Render permite automatizar parte del flujo de despliegue, reduciendo diferencias entre el código almacenado y la versión publicada. Para un proyecto integrador, esta evidencia del proceso de desarrollo es tan relevante como el código final.

16. Expo EAS Build para Android

Para generar el artefacto Android se utiliza Expo EAS Build. Esta plataforma simplifica la compilación y firma del APK, reduciendo problemas derivados de configurar manualmente todos los componentes del entorno Android en cada equipo de desarrollo. Para un prototipo académico esto disminuye tiempo de configuración y permite obtener

una versión instalable y reproducible en dispositivos físicos. Al mismo tiempo, Expo permite generar o acceder al proyecto nativo cuando se requieren ajustes específicos, por lo que esta decisión no elimina la posibilidad de evolución técnica posterior.

17. Proporcionalidad entre alcance y complejidad

Una de las decisiones más importantes fue no utilizar una arquitectura más compleja de la necesaria. Barbería Cale v1.0 no requiere microservicios, múltiples bases de datos, balanceadores, colas distribuidas ni infraestructura empresarial porque su dominio está concentrado en autenticación y gestión de citas y el tráfico previsto es reducido. Introducir esas tecnologías aumentaría costos, puntos de fallo, mantenimiento y conocimiento requerido sin resolver un problema presente. Sin embargo, tampoco se eligió una estructura completamente monolítica donde las pantallas acceden directamente a la base de datos. La arquitectura actual representa un punto intermedio: suficientemente organizada para seguridad, mantenimiento y evolución, pero compatible con el alcance, presupuesto y capacidad técnica disponibles.

18. Criterio global de selección tecnológica

En conjunto, las decisiones se tomaron buscando equilibrio entre alcance funcional, costo, compatibilidad multiplataforma, tiempo de desarrollo, capacidad técnica, mantenibilidad, seguridad y escalabilidad futura. La prioridad de la versión 1.0 es resolver correctamente el problema real de gestión de citas con una arquitectura que el equipo pueda implementar, probar, documentar y mantener. Al mismo tiempo, las principales tecnologías seleccionadas son estándares ampliamente utilizados y desacoplados entre sí, lo que permite sustituir infraestructura o ampliar funcionalidades sin reconstruir completamente el sistema. Si Barbería Cale evolucionara hacia múltiples barberos,

sucursales, pagos, notificaciones automáticas o una carga mucho mayor, la separación actual proporcionaría una base sobre la cual realizar esa evolución de forma gradual.